

DECENTRALIZED AUCTION SYSTEM

Team Hyperledgerz

Name		Roll No
Bhavika Jaiswal	-	240001017
Disha Dange	-	240001026
Rida Samrin	-	240001060
Sanskriti Jain	-	240001064
Shriya Deo	-	240041013
Siddhi Patil	-	240041035

1. Problem Statement

1.1 The Limitations of Traditional Auctions

Traditional auction systems, whether conducted in physical venues or through centralised online platforms, suffer from a range of structural and operational shortcomings that undermine trust, transparency, and access for participants.

1.1.1 Core Problems in Existing Auction Systems

- **Opacity and Information Asymmetry:** In most conventional auctions, the history of bids, the identity of bidders, and the mechanics of price discovery are opaque. Only the auctioneer holds complete information, creating an inherent information asymmetry that can be exploited through practices such as phantom bidding, bid shielding, or shill bidding — where fictitious bids are placed to artificially inflate prices.
- **Intermediary Dependency and Associated Costs:** Centralised auction platforms — eBay, Christie's, Sotheby's online portals — require trusted third-party intermediaries to custody funds, verify identities, and enforce settlement. These intermediaries charge fees ranging from 5% to 25% of the final sale price, and their continued operation depends on institutional trust that can be misplaced.
- **Settlement Risk and Payment Failures:** In traditional auctions, the winning bidder must pay after the fact, often through bank transfers or cheques. This creates settlement risk — the possibility that the winning bidder defaults, requiring re-auctioning, legal enforcement, or financial loss to the seller.
- **Geographic and Access Barriers:** Physical auction houses exclude global participants. Even online platforms impose geographic restrictions through payment gateways, currency conversion costs, and regulatory barriers, artificially limiting the pool of potential buyers and depressing competitive price discovery.
- **Fraud and Dispute Resolution:** Disputes about whether bids were validly placed, whether the auction rules were followed, or whether the seller fulfilled their obligations are resolved through centralised arbitration that is slow, costly, and potentially biased. There is no immutable record that both parties can independently verify.

1.2 Why Blockchain is the Right Tool

Blockchain technology, specifically the Ethereum smart contract platform, directly addresses each of the above failure modes through a combination of technical properties that are uniquely suited to auction mechanics.

1.2.1 Trustless Execution

A smart contract deployed on Ethereum executes deterministically and autonomously. Once the auction code is deployed, no party — including the contract deployer — can alter its logic mid-auction. The rules encoded in the contract are the only rules. This eliminates the need to trust an intermediary, replacing institutional trust with cryptographic certainty.

1.2.2 Transparent and Immutable Bid History

Every bid placed on-chain is permanently recorded on the Ethereum blockchain and is publicly verifiable. The `BidPlaced` event emitted by the contract logs the auction identifier, bidder address, and net bid amount. Any participant can independently audit the entire bid history using a block explorer, making phantom bidding and bid manipulation cryptographically impossible.

1.2.3 Atomic Settlement with Locked Funds

When a bidder calls `placeBid()`, their ETH is transferred to the contract in the same transaction. Funds are locked in escrow by the contract itself. The winning amount is only released to the seller upon calling `endAuction()` and then `withdrawBid()`. Losing bidders can retrieve their funds at any time after being outbid. This pull-payment model eliminates settlement risk entirely — funds are already in the contract before the auction closes.

1.2.4 Permissionless Global Access

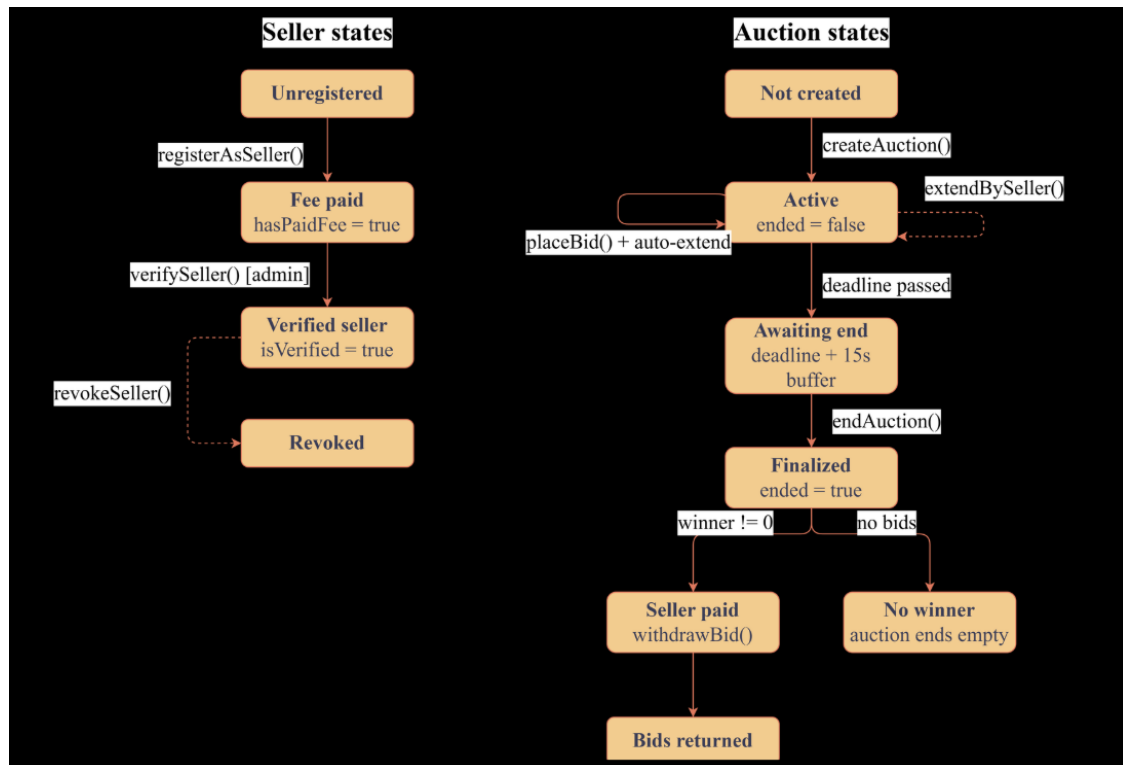
Any Ethereum address, regardless of geography, nationality, or banking relationship, can participate in the auction provided they hold sufficient ETH. This creates a truly global, permissionless marketplace that maximises competitive price discovery by including the widest possible pool of bidders.

1.2.5 Anti-Manipulation Mechanisms

The contract implements an anti-sniping mechanism: if a bid is placed within the final 3 minutes of the auction, the deadline is automatically extended by 3 minutes (up to 5 times). This prevents the well-known strategy of bid sniping — placing a winning bid in the final seconds to prevent competing bids — and ensures a genuinely competitive close. This logic is enforced in code, not policy.

2. Architecture

Architecture Diagram



Architecture Diagram

2.1 System Overview

The Auction contract is a single-file, self-contained Solidity contract inheriting from OpenZeppelin's ReentrancyGuard. It manages the full lifecycle of multiple concurrent auctions through a mapping-based data structure, with separate concern areas for seller management, auction lifecycle, bidding mechanics, fund custody, and administrative controls.

State Variable	Type	Purpose
auctions	mapping(uint256 => AuctionDetails)	Stores all auction data keyed by sequential ID
auctionCounter	uint256	Auto-incrementing primary key for auctions
pendingReturns	mapping(uint256 => mapping(address => uint256))	Escrow ledger: stores refundable bid amounts per auction per bidder
_hasBid	mapping(uint256 => mapping(address => bool))	Tracks unique bidder participation for fee calculation

sellers	mapping(address => SellerInfo)	KYC/registration status for each seller address
registeredSellers	address[]	Enumerable list of all registered sellers
accumulatedFees	uint256	Total platform fees collected, withdrawable by owner

2.2 Function Architecture and Relationships

The diagram provided shows the complete state machine. The following table describes each function and its role in the system:

Function	Access	Category	Description
registerAsSeller()	public payable	Seller Lifecycle	Pays registration fee (0.01 ETH), sets hasPaidFee = true, adds to registeredSellers array
verifySeller(addr)	onlyOwner	Seller Lifecycle	Admin grants isVerified = true to a fee-paying seller, enabling auction creation
revokeSeller(addr)	onlyOwner	Seller Lifecycle	Admin revokes verification, preventing future auctions (does not affect active auctions)
isVerifiedSeller(addr)	view	Seller Lifecycle	Returns current verification status
getRegisteredSellersCount()	view	Seller Lifecycle	Returns length of the registeredSellers array
createAuction(...)	verified seller	Auction Lifecycle	Creates new auction with IPFS CID, starting price, duration (10min-24h), and minimum increment
extendBySeller(id, time)	auction seller	Auction Lifecycle	Allows the seller to manually extend deadline, capped at 20% of original duration
endAuction(id)	public	Auction Lifecycle	Finalises auction after deadline + 15s buffer; credits highest bid to seller's pendingReturns

placeBid(id)	public payable	Bidding	Places a bid: deducts platform fee, enforces increment rules, triggers anti-snipe extension if needed
withdrawBid(id)	public nonReentrant	Fund Management	Pull-payment withdrawal: bidders reclaim outbid amounts; seller claims winning bid after end
buyerFee(id)	view	Fee Calculation	Returns dynamic fee = $\text{baseBuyerFee} + (\text{numBidders} * 0.0001 \text{ ETH})$
minimumBidTotal(id)	view	Fee Calculation	Returns total ETH a bidder must send: net bid required + current buyer fee
getAuction(id)	view	Queries	Returns core auction details: CID, seller, highestBid, highestBidder, deadline, ended, numBidders
timeRemaining(id)	view	Queries	Returns seconds until deadline, or 0 if passed
updateFees(...)	onlyOwner	Administration	Updates registration and base buyer fee parameters
withdrawFees()	onlyOwner nonReentrant	Administration	Owner withdraws all accumulated platform fees

2.3 Fee Architecture

The contract implements a two-tier fee model designed to discourage auction spam and reward early participation:

- **Seller Registration Fee (fixed):** 0.01 ETH, paid once to register. This creates a financial barrier against Sybil attacks on the seller side. The fee is non-refundable, even if verification is denied, although the contract does not enforce an approval deadline.
- **Buyer Fee (dynamic):** $\text{baseBuyerFee} (0.001 \text{ ETH default}) + (\text{numBidders} * 0.0001 \text{ ETH})$. The fee increases with each unique bidder, penalising late entrants and compensating the platform for the additional state management. This is deducted from each bid's msg.value; only the net amount (msg.value - fee) is recorded as the actual bid.

The minimumBidTotal() view function combines the required net bid with the current buyer fee, giving front-end interfaces a single figure to display to users.

3. Security Analysis

The contract addresses the common Solidity vulnerability classes through a combination of OpenZeppelin guards, the Checks-Effects-Interactions pattern, and explicit access control. The table below summarises each threat and its mitigation, followed by brief notes on the key mechanisms.

Attack / Risk	Status	Mechanism
Reentrancy	Prevented	nonReentrant on withdrawBid() + withdrawFees(); CEI pattern zeroes state before ETH transfer
Bid sniping	Mitigated	Late bid (< 3 min remaining) auto-extends deadline by 3 min, up to 5 times
Seller deadline abuse	Prevented	Total seller extensions capped at 20% of original auction duration
Shill / self-bidding	Prevented	placeBid() reverts with SellerCannotBid if msg.sender == auction.seller
Double-bid by leader	Prevented	AlreadyHighestBidder revert if msg.sender == highestBidder
Griefing / DoS via reverting fallback	Prevented	Pull-payment model — no ETH is ever pushed; each party withdraws themselves
Integer overflow / underflow	Prevented	Solidity ^0.8.20 applies automatic checked arithmetic on all operations
Timestamp manipulation	Mitigated	endAuction() requires block.timestamp >= deadline + 15 s buffer
Unauthorised admin actions	Prevented	onlyOwner modifier; all admin calls revert with NotOwner for non-owner callers
Front-running	Residual risk	No commit-reveal scheme; front-runners must still outbid, which is normal market behaviour

3.1 Key Mechanisms

Reentrancy guard. withdrawBid() follows the CEI pattern: it records the amount, zeroes pendingReturns[auctionId][msg.sender], then makes the external call. The nonReentrant modifier provides a second layer of defence.

Anti-sniping. Implemented directly inside placeBid(). If deadline - block.timestamp < EXTENSION_TIME and extensionCount < MAX_EXTENSIONS, the deadline extends by 3

minutes and extensionCount increments. The cap of 5 extensions limits maximum snipe protection to 15 minutes.

Pull payments. All ETH movements — outbid refunds, seller proceeds, and admin fees — sit in pendingReturns or accumulatedFees until the entitled party calls a withdrawal function. No contract function pushes ETH to an external address unsolicited, eliminating the reverting-fallback griefing vector.

Access control. Three tiers: owner-only (onlyOwner modifier), seller-only (inline msg.sender == auction.seller check in extendBySeller), and verified-seller-only (isVerified check in createAuction). All sensitive state transitions revert with a descriptive custom error on failure.

4. Gas Optimization

Unoptimized gas report (forge test--gas-report) :

src/Auction.sol:Auction Contract					
Deployment Cost	Deployment Size				
1677881	7205				
Function Name	Min	Avg	Median	Max	# Calls
buyerFee	7371	7371	7371	7371	7
createAuction	236748	236748	236748	236748	5
endAuction	35427	48771	48771	62116	2
getAuction	18165	18165	18165	18165	1
placeBid	46884	99948	111795	111795	7
registerAsSeller	91390	91390	91390	91390	5
verifySeller	30503	30503	30503	30503	5
withdrawBid	38281	38281	38281	38281	2

Highest gas consuming function : createAuction()

Analysis - why it consumed max gas :

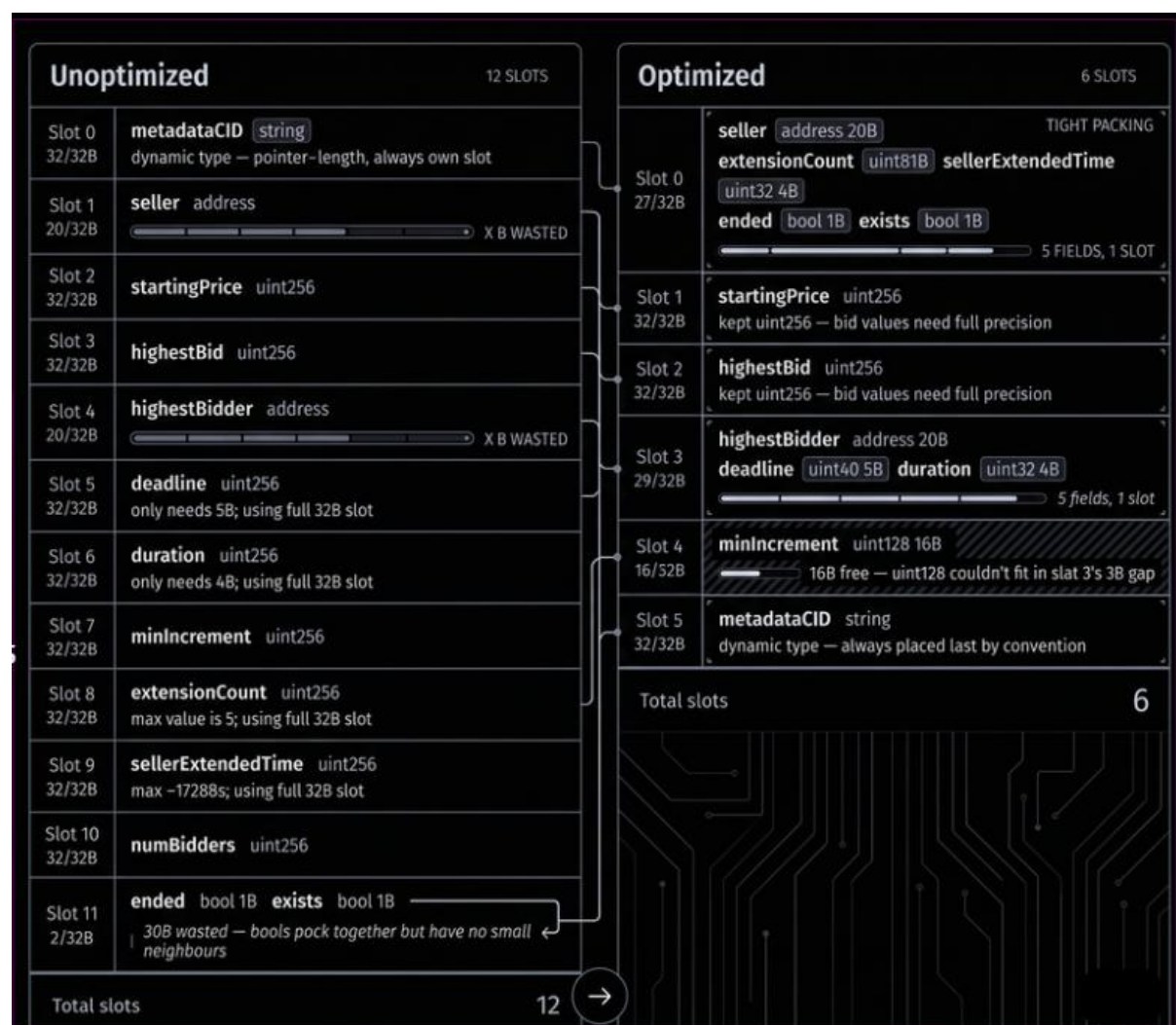
EVM manages storage in 32-byte slots. Each time a new auction is initialized, the contract uses the SSTORE opcode-the most expensive operation in Solidity to write data permanently to the blockchain.

The EVM fills these storage slots in the exact order that variables are declared in the code. If small variables (like bool or address) are separated by large 32-byte variables (like uint256),

the EVM cannot group them together. Instead, it is forced to waste the unused space in the current slot and start a new 32-byte slot for the next variable. Since each new slot costs 20,000 gas to initialize, this gap in storage significantly increases the total gas cost of the function.

Solution : Variable packing

This involves organizing the struct to maximize storage efficiency. By using downcasting—choosing smaller data types like uint64 or uint8 for variables like duration or extensionCount that are guaranteed to fit in less space—we grouped multiple variables together within a single 32-byte slot. This strategic ordering allowed the EVM to consolidate several pieces of data into one SSTORE operation instead of multiple expensive ones. Ultimately, this approach ensured that the contract uses the minimum number of storage slots possible, significantly lowering the gas cost for every new auction created.



Other Optimizations :

- **Calldata vs Memory :**

In the original contract, dynamic inputs like string itemName were stored in memory. This caused the EVM to copy data from calldata into memory, increasing gas usage.

The contract was updated to use calldata for external function parameters. Since calldata is read-only and already holds the input data, this removes the need for copying and reduces gas costs for functions like createAuction().

- **Custom Errors :**

require statements with long error messages were replaced with custom errors.

Lower deployment cost: Long strings increased contract size, making deployment more expensive.

Lower execution cost: Custom errors used a short 4-byte selector instead of full strings, saving gas when transactions revert.

Overall, these changes reduced both deployment and execution gas costs.

Optimized gas report (forge test--gas-report) :

src/Auction.sol:Auction Contract					
Deployment Cost	Deployment Size				
1458793	6187				
Function Name	Min	Avg	Median	Max	# Calls
buyerFee	7311	7311	7311	7311	7
createAuction	186080	186080	186080	186080	5
endAuction	33253	44482	44482	55711	2
getAuction	11913	11913	11913	11913	1
placeBid	40544	83714	88206	96317	7
registerAsSeller	91390	91390	91390	91390	5
verifySeller	30497	30497	30497	30497	5
withdrawBid	38175	38175	38175	38175	2

- Overall deployment cost reduced from 1677881 to 1458791, saved 219090
- createAuction() cost reduced from 236748 to 186080 saved 50668

4. Coverage Report

Metric	Coverage
Line Coverage	92.24%
Statement Coverage	84.83%
Function Coverage	100%
Branch Coverage	62.16%

Overall Project Coverage

Metric	Coverage
Total Line Coverage	87.41%
Total Statement Coverage	82.72%
Total Function Coverage	89.29%
Total Branch Coverage	60.98%

File	% Lines	% Statements	% Branches	% Funcs
script/Counter.s.sol	0.00% (0/5)	0.00% (0/3)	100.00% (0/0)	0.00% (0/2)
src/Auction.sol	92.24% (107/116)	84.83% (123/145)	62.16% (23/37)	100.00% (18/18)
src/Counter.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
src/ReentrancyAttacker.sol	100.00% (9/9)	100.00% (6/6)	100.00% (2/2)	100.00% (3/3)
test/ReentrancyAttacker.sol	55.56% (5/9)	50.00% (3/6)	0.00% (0/2)	66.67% (2/3)
Total	87.41% (125/143)	82.72% (134/162)	60.98% (25/41)	89.29% (25/28)